現代的Web開発・基礎勉強会

Section 02: ロジックの構築とデータ構造

1. Section 01 の振り返り

前回は JavaScript のルールと「材料」を学びました。

- **JS の特徴**: Java とは別物。型が曖昧（動的型付け）。
- **変数と型**: `const` / `let` の使い分け。2つの「ない」（`null`, `undefined`）。
- **ロジック**: `===` による厳密比較。7つの **falsy** な値。
- **演算子**: `&&`, `||`, `??` によるショートサーキット評価。
- **全体像**: Frontend / Backend / そしてそれらを繋ぐ **API**。

今回は、これらを組み合わせて **「複雑なロジック」** や **「データの集合」** を扱う方法を学びます。

2. 制御構文 (Control Flow)

プログラムは一本道ではありません。「もし在庫があれば」「もしエラーなら」といった**分岐**が必要です。

2-1. 基本の if 文

「もし〜なら、〜する」という最も基本的な分岐です。

```
const score = 85;

// 条件式が true の場合のみ実行される
if (score >= 80) {
  console.log("合格!おめでとう");
} else {
  // 条件式が false の場合
  console.log("不合格...再挑戦しよう");
}
```

> これは読みやすいですが、条件が増えると行数が長くなりがちです。

2-2. 三項演算子 (Ternary Operator) : 概念

Web開発では **「値として結果が欲しい」** 場面が発生します。
`if文` は「文（命令）」なので、特定の変数に結果を入れるには行数が必要です。

If文 vs 三項演算子

- **if文**: 「もし男なら太郎という名前をつけ、女なら花子という名前**をつける**」
- **三項演算子**: 「男なら「太郎」、女なら「花子」という名前**にする**」

2-4. 「値として欲しい」とは？

プログラミングには **「文 (Statement)」** と **「式 (Expression)」** という違いがあります。

`if` の結果は「動作」

結果によって「何かをする」ことしかできず、「変数に代入」できません。

```
// エラー! if文自体は「値」ではない
const result = if (score > 80) { "合格" } else { "不合格" };
```

三項演算子 は「式」

「計算結果（値）」になるので、変数に入れたり、そのまま使えます。

```
// OK! 式全体が "合格" という値に変身するイメージ
const result = score > 80 ? "合格" : "不合格";
```

モダンなJavaScriptでは、`const` に値を一発で入れるために三項演算子がよく使われます。

2-3. 三項演算子：構文

「条件 ? Trueの時 : Falseの時」

`if-else` を1行で書くための構文です。

```
const isMember = true;

// 従来の書き方（行数を使う）
// let fee;
// if (isMember) { fee = 1000; } else { fee = 2000; }

// 三項演算子（スッキリ!）
const fee = isMember ? 1000 : 2000;
```

右側（`:` の後ろ）を書き忘れるミスが多いので注意しましょう。

練習問題: 三項演算子による条件分岐

「オンライン(`isOnline`)」なら `"Online"` 、そうでなければ `"Offline"` という文字列を代入したいです。
三項演算子の空欄を埋めてください。

```
const isOnline = true;
const statusMessage = isOnline ____ "Online" ____ "Offline";
```

【解説】 三項演算子による条件分岐

```
const statusMessage = isOnline ? "Online" : "Offline";
```

`条件 ? Trueの値 : Falseの値` という構文です。短い `if-else` 文はこれで一行にスッキリ書けます。

2-4. 三項演算子：実践的な活用

画面に表示するメッセージや、設定値を切り替えるのによく使います。

```
const role = "admin";

// 権限に応じてメッセージを変える
const message = role === "admin" ? "管理者設定へ進む" : "閲覧のみ可能です";

console.log(message);
```

「条件に応じて中身がコロコロ変わる変数」を作るのに最適です。

Q1. 三項演算子の結果は？

以下のコードを実行した時、`message` には何が入るでしょうか？

```
const user = null;
const message = user ? "ログイン済み" : "ゲスト";
console.log(message);
```

1. `"ログイン済み"`
2. `"ゲスト"`
3. `undefined`
4. エラーになる

Q1. 解説

正解: 2. `"ゲスト"`

前パートで学んだ **Falsyな値** のルールに基づきます。
`user` が `null` の場合、条件式では `false` と判定されるため、右側の `"ゲスト"` が返されます。

> **現場での活用**:
> APIからデータが届くまでは `null` にしておき、`user ? <表示> : <読み込み中>` のように切り替えるパターンが非常に多いです。

2-5. オプショナルチェーン (Optional Chaining) : `?.`

「もしかしたら無いかもしれない」データに安全にアクセスできます。

```
const user = {};

//  エラーにならず、undefined が返る
console.log(user.profile?.name);
```

「もしあれば進む、なければそこで止まる」魔法の記号です。

3. 関数 (Functions)

3-0. 関数とは？

関数は、**特定の処理をひとまとめにしたもの** です。
材料（データ）を入れて、加工して、結果を取り出します。

- **入力 (Input)**: ミキサーに「果物」を入れる → **引数**
- **処理 (Process)**: 中で「細かく砕く」
- **出力 (Output)**: 「ジュース」が出てくる → **戻り値**

一度作っておけば、中身を気にせず **「名前を呼ぶだけ」** で何度でも使えます。

3-1. 基本の書き方（`function`）

まずは最も伝統的な `function` キーワードを使った書き方です。

```
// 1. 定義する
function sayHello(name) {
  return `こんにちは、${name}さん!`;
}

// 2. 実行する(呼び出す)
const result = sayHello("Shake");
console.log(result); // "こんにちは、Shakeさん!"
```

- `name`：**引数**（外から受け取る材料）
- `return`：**戻り値**（外へ返す結果）

3-2. 関数のメリット

1. **再利用性**: 同じコードを何度も書かなくて済む。
2. **部品化 (Modularization)**: 複雑なプログラムを、小さな部品に分けて管理できる。
3. **保守性**: 不具合があった時、その関数の「中身」だけ直せば全体が直る。

3-3. 関数の進化とアロー関数

昔: Function Declaration

```
function add(a, b) {
  return a + b;
}
```

「関数宣言」です。巻き上げ（定義より前で使える）などの挙動がありますが、記述が長くなりがちです。

今: Arrow Functions (Modern)

```
const add = (a, b) => {
  return a + b;
};
```

ES6(2015)で登場。変数に代入する形ですっきり書けます。
現代のJSではこちらが主流です。

3-4. アロー関数の構文バリエーション

アロー関数には「省略記法」があり、これが多用されます。

基本形

```
const double = (num) => {
  return num * 2;
};
```

省略形 (Implicit Return)

処理が1行で、値を返すだけなら `{}` と `return` を消せます。

```
const double = (num) => num * 2;
```

「即座に(implicitly)値を返す」という意味です。これがコードを劇的に短くします。

Q2. アロー関数の戻り値

以下の関数 `getPrice` を実行した結果は？

```
const getPrice = (price) => {
  price * 1.1;
};
console.log(getPrice(100));
```

1. `110`
2. `100`
3. `undefined`
4. `null`

Q2. 解説

正解: 3. `undefined`

落とし穴です！

`{}` (波括弧) を書いた場合、明示的に `return` を書かないと戻り値は `undefined` になります。

```
// 正しい書き方 A (returnあり)
const getPrice = (price) => {
  return price * 1.1;
};

// 正しい書き方 B (Implicit Return: 波括弧なし)
const getPrice = (price) => price * 1.1;
```

練習問題: アロー関数の省略記法

引数を2乗して返す関数 `square` を定義したいです。
アロー関数の `____` を埋めて、最も短い書き方で完成させてください。

```
// 値をそのまま返す (Implicit Return)
const square = (x) => ____;
```

【解説】 アロー関数の省略記法

```
const square = (x) => x * x;
```

処理が式ひとつだけの場合、`{}` と `return` を省略できます。
`{ return x * x; }` と書いても正解ですが、省略記法の方がスマートです。

3-5. 関数は「データ」として扱える

JavaScript では、**関数（処理）を「数値」や「文字列」と同じように変数に入れることができます。**

```
// 数値を変数に入れる(データ)
const score = 100;

// 関数を変数に入れる(ロジック)
const greet = () => {
  console.log("こんにちは!");
};
```

変数に入れたものは、**他の関数に「引数」として渡すことができます。**

3-5-1. コールバック関数とは？

「あとで実行してもらうために、**関数（処理そのもの）を他の関数に預ける**」ことをコールバックと呼びます。

- **数値の受け渡し**: 「100 という **データ** をあげる」
- **関数の受け渡し**: 「この **やり方（ロジック）** でやっておいてね、と託す」

3-5-1. よくある使われ方 (1) 待機処理

「何かを待ってから実行する」というパターンです。
Node.js やブラウザなど、あらゆる JS 環境で利用されます。

```
// 「1秒(1000ミリ秒)待ってから、この関数を呼び出して」と託す
const showMessage = () => {
  console.log("1秒経過しました!");
};

setTimeout(showMessage, 1000);
```

3-5-2. よくある使われ方 (2) 処理の完了報告

「重い処理（データの読み込みなど）が終わったら、これをやって！」という約束に使います。

```
// 読み込みが終わった時に実行してほしい「挨拶」を渡す
const onFinish = (name) => {
  console.log(`${name}さんのデータの読み込みが終わりました!`);
};

// 本来はここで重い処理をする
const dataLoading = (callback) => {
  const userName = "Shake";
  callback(userName); // 終わったので、預かっていた関数(挨拶)を実行
};

dataLoading(onFinish);
```

3-6. なぜアロー関数が好まれるのか？

単に「短いから」だけではありません。**「コールバック関数」として使いやすい** からです。
別の関数に「やり方」を預ける時、アロー関数なら1行で直感的に書くことができます。

```
// 従来の書き方
setTimeout(function () {
  console.log("1秒経過");
}, 1000);

// アロー関数
setTimeout(() => console.log("1秒経過"), 1000);
```

4. 配列 (Arrays) とその操作

配列とは、同じ種類のデータが順番に並んだ **「データの連なり（リスト）」** です。
これからの操作で頻繁に登場する **「メソッド (Method)」** という言葉を覚えておきましょう。

- **メソッド**: そのデータ（配列など）に最初から備わっている「専用の関数」のこと。
- **呼び出し方**: `データ.メソッド名()` のように、ドット（`.`）を付けて呼び出します。

現代のJavaScriptでは `for` ループよりも、この **配列専用のメソッド** を使うのが一般的です。

4-1. 配列の基本

まずは「入れ物」としての使い方を覚えましょう。

書き方とアクセス

```
const fruits = ["りんご", "バナナ", "みかん"];

// 0番目から数える（重要!）
console.log(fruits[0]); // "りんご"
console.log(fruits[1]); // "バナナ"

// データの数を知る
console.log(fruits.length); // 3
```

- プログラミングでは「1から」ではなく **「0から」** 数えるのがルールです。

4-2. 配列の操作 (追加・削除)

配列の中身を直接書き換えるメソッドの基本です。

```
const list = ["A", "B"];

// 末尾に追加
list.push("C");
console.log(list); // ["A", "B", "C"]

// 末尾を削除
list.pop();
console.log(list); // ["A", "B"]
```

これらのメソッドは **「元のデータを直接書き換える」** という特徴があります。

4-3. 繰り返し処理 (1)：for / while

データをひとつずつ取り出して処理する「ループ」の方法です。

for 文（定番）

回数を指定して繰り返します。

```
for (let i = 0; i < 3; i++) {
  console.log(`${i}番目のループ`);
}
```

while 文（条件が合うまで）

条件が true の間、ずっと繰り返します。

```
let count = 0;
while (count < 3) {
  console.log("ループ中...");
  count++;
}
```

4-4. 繰り返し処理 (2)：forEach

配列のデータをすべて処理したい時に使います。
`for` 文よりも「配列を扱う」という意図が明確になります。

```
const colors = ["赤", "青", "緑"];

// colors の中身をひとつずつ color という名前で取り出す
colors.forEach((color) => {
  console.log(`${color}を塗ります`);
});

/*出力
赤を塗ります
青を塗ります
緑を塗ります
*/
```

「関数を引数に渡す」という、先ほどのコールバックの形になっていますね。

4-5. Mapメソッド (.map) : 工場のイメージ

.map() は「変換工場」です。

ベルトコンベア（配列）に乗って流れてくる材料を、一つずつ加工して、新しい製品（新しい配列）として送り出します。

- **入力**: `[🍎, 🍌, 🍊]` （素材の配列）
- **処理**: `ミキサーにかける(ジュースにする)`
- **出力**: `[🥤, 🥤, 🥤]` （結果の配列）

「数は変わらないが、中身の形が変わる」 のが .map() の最大の特徴です。

4-6. Mapメソッド (.map) : 実装

```
const prices = [100, 200, 300];

// 全商品を1.1倍(税込)にする
const taxPrices = prices.map((price) => price * 1.1);
// [110, 220, 330] になる
```

実践パターン

「元のデータを一括で変換して、新しいリストを作る」際によく使います。

```
const kmList = [1, 5, 10];

// すべてをメートル(m)に変換する(1000倍する)
const mList = kmList.map((km) => km * 1000);
// [1000, 5000, 10000]
```

練習問題: Mapメソッドによる変換

数値の配列 `numbers` があります。これらをすべて「2倍」にした新しい配列 `doubled` を作りたいです。
`.map` の中身（コールバック関数）を書いてください。

```
const numbers = [1, 2, 3];

// 各要素 n を 2倍にする
const doubled = numbers.map((n) => ____);
```

【解説】 Mapメソッドによる変換

```
const doubled = numbers.map((n) => n * 2);
```

`.map` は「配列の要素ひとつひとつを変換して、新しい配列を作る」メソッドです。Reactなどでリストを表示する際にも頻繁に使います。

4-7. Filterメソッド (.filter) : 入場ゲートのイメージ

.filter() はいわば「選別機」です。

条件（チケットの有無や、腐っていないかなど）に合致する要素だけを通し、合わないものを弾きます。

- **入力**: `[🍎, 🍕, 🍎, 🍔]` （食べ物の配列）
- **処理**: `フルーツ(🍎)だけを通す`
- **出力**: `[🍎, 🍎]` （結果の配列）

「中身の形は変わらないが、数は減る（または同じ）可能性がある」 のが .filter() の特徴です。

4-8. Filterメソッド (.filter) : 実装

```
const scores = [50, 90, 40, 80];

// 60点以上だけ合格リストに入れる
const passed = scores.filter((score) => score >= 60);
// [90, 80] だけが残る
```

用途:

- 完了したタスクをリストから除外する
- 特定の条件に合致するデータだけ抽出する

Q3. 配列操作の使い分け

「価格リストから**300円以下のものだけ**を取り出したい」。
使うべきメソッドは？

```
const prices = [100, 500, 200, 800];
const cheapPrices = prices.____((p) => p <= 300);
```

1. `.map()`
2. `.filter()`
3. `.push()`
4. `.forEach()`

Q3. 解説

正解: 2. `.filter()`

- **Filter**: 条件に合うものを「残す」。数が減る可能性がある。
- **Map**: 全員を「変形させる」。数は変わらない。

「リスト変換」ならMap、「絞り込み・検索」ならFilter、と覚えましょう。

4-9. メソッドチェーン (組み合わせ)

これらのメソッドは繋げることができます。
「500円以下の商品だけを、すべて10%引きにする」という場合：

```
const prices = [100, 600, 400, 800];

const result = prices
  .filter((p) => p <= 500) // まず「500円以下」で絞り込む ([100, 400])
  .map((p) => p * 0.9); // 次に「10%引き」に計算する

console.log(result); // [90, 360]
```

処理の流れが左から右へ（上から下へ）流れるように読めるのが特徴です。

4-10. その他の重要メソッド (.find, .includes)

.find() : 探索

「条件に合う最初のひとつを探す」

配列全体ではなく、特定の1件（例：IDが一致するユーザー）が欲しい時に使います。
戻り値は配列ではなく、**要素そのもの**（見つからなければ `undefined` ）です。

.some() : 条件を満たすものがあるかの検索

「条件を満たすものが一つでもあるかチェックする」

単純に `true` か `false` が返ります。
「許可リストに入っているか？」などの権限チェックによく使います。

4-10. 不変性 (Immutability)：ノートの例え

モダンなJS開発において、最も大切なルールは **「元のデータを直接書き換えない」** ことです。

なぜ `push` はダメなのか？ (副作用)

`push` は「みんなで共有しているノート」に勝手に追記するようなものです。

- あなたがページを増やしたせいで、他の人が「このノートは3ページのはずだ」と思って計算していた処理がバグってしまいます。これを **副作用 (Side Effect)** と呼びます。

解決策：不変性 (Immutability)

「元のノートには触らず、コピー（新しい配列）を取ってから、そこに書き足す」 のが鉄則です。

4-12. 不変性を守る操作：スプレッド構文

「元の配列をコピーした新しい配列を作り、それを変数に上書きする」ことで安全にデータを更新します。

```
const list = ["A", "B"];

//  共有ノートに直接書き込む（他の場所でバグの原因に）
// list.push("C");

//  コピーを取って新しいノートを作る
const newList = [...list, "C"];
```

元の `list` は綺麗なままなので、どこで使っても安全な「信頼できるデータ」になります。

練習問題: スプレッド構文による要素追加

既存の `fruits` 配列をコピーしつつ、新しい要素 `"メロン"` を末尾に追加した `newFruits` を作りたいです。

```
const fruits = ["りんご", "バナナ"];

// fruits を展開して、新しく "メロン" を追加
const newFruits = [____, "メロン"];
// → ["りんご", "バナナ", "メロン"] の形式にしたい
```

【解説】 スプレッド構文による要素追加

```
const newFruits = [...fruits, "メロン"];
```

`...fruits` で元の配列のなかみを展開します。元の配列 `fruits` は変更されず（不変性）、新しい配列が作られます。

4-11. 不変性がないとどうなる？ (バグの実例)

「参加者リストに自分を追加しただけなのに、他のリストまで変わってしまった」というバグです。

```
// 1. Aチームのリスト（原本）
const teamA = ["田中", "佐藤"];

// 2. Bチームのリスト（本来はコピーすべきだが、代入してしまう）
const teamB = teamA;

// 3.  直接書き換える（Mutation）
teamB.push("自分"); // Bチームに自分を入れたつもり

// 4. Aチームまで書き換わっている!
console.log(teamA); // ["田中", "佐藤", "自分"]
```

結論: 配列（や、次で学ぶオブジェクト）を直接いじると、アプリのあちこちで「データが勝手に変わる」という呪いのようなバグが発生します。だから **不変性** は必須なのです。

5. オブジェクト (Objects) のモダンな扱い方

名前付きデータの集合体「オブジェクト」について扱います。
ここでも「いかに簡潔に書くか」「いかに安全に書くか」が進化しています。

5-0. 基本用語：キーとバリュー

オブジェクトを扱う上で必須の言葉です。**「名前付きのハコ」** をイメージしてください。

- **キー (Key)**: データの「名前（ラベル）」
  - 例: `name`, `age`, `isAdmin`
- **バリュー (Value)**: そのハコの中身（実体）
  - 例: `"Shake"`, `20`, `false`

「どのラベル（キー）に、どんな中身（バリュー）が入っているか」のセットを並べたものがオブジェクトです。

5-1. JSON (JavaScript Object Notation)

JSONは、 **「データを届けるための共通ルール（フォーマット）」** です。
プログラム同士でデータを送るためのルールだと考えてください。

「オブジェクト」と「JSON」

初心者の方が最も混乱しやすいのがこの違いです。

特徴	JavaScript オブジェクト	JSON (文字列)
正体	**メモリ上の「実体」**	**ただの「テキスト」**
用途	プログラムの中で計算・操作する	通信や保存のために書き出す
コード例	`const user = { age: 20 };`	`const json = '{ "age": 20 }';`
操作	`user.age` で中身が読める	テキストなので中身は読めない

データの受け渡しフロー：パッキングと組み立て

1. **JSON.stringify() : パッキング (文字列化)**
   - 送る側：手元の「生きたデータ」を、送信用に「テキスト」へ書き出す。
   - `const json = JSON.stringify(obj);`
2. **送信**
   - ネットワークを通じて、ただのテキストとして相手に届く。
3. **JSON.parse() : 組み立て (復元)**
   - 受け取る側：届いた「テキスト」を、自分の手元で「生きたデータ」に組み立て直す。
   - `const obj = JSON.parse(json);`

5-1-1. JSONの具体例：よくあるパターン

構文ルールは **「キーを必ず `" "` (ダブルクォート) で囲む」** こと。非常に厳格です。

1. 単体のデータ (User)

```
{
  "id": 101,
  "name": "田中 太郎",
  "isAdmin": false
}
```

2. リストデータ (Products) - [ ] 配列が含まれる

```
[
  { "id": 1, "name": "PC", "price": 100000 },
  { "id": 2, "name": "Mouse", "price": 3000 }
]
```

5-1-2. JSONの具体例：高度な構造（ネスト）

複雑なデータも、オブジェクトの中にオブジェクト（または配列）を入れることで表現できます。

3. 入れ子構造 (Nested)

```
{
  "event": "学園祭",
  "location": "A棟ホール",
  "schedule": [
    { "time": "10:00", "title": "開会式" },
    { "time": "11:00", "title": "LIVE" }
  ],
  "contact": {
    "email": "info@example.com",
    "tel": "03-xxxx-xxxx"
  }
}
```

5-1-3. まとめる：JSONで扱える型

JSONでは、プログラムで使う主要なデータをすべて表現できます。

型	例	備考
文字列	`"Hello"`	必ずダブルクォート
数値	`123`, `0.5`	そのまま書く
真偽値	`true`, `false`	そのまま書く
Null	`null`	空であることを示す
配列	`[...]`	データのリスト
オブジェクト	`{...}`	名前付きデータの集合

5-1-4. JSONの具体例：SNSの投稿データ（入れ子構造）

「全体がひとつのオブジェクト」であり、その中に「また別のオブジェクトや配列」が入る構成です。

```
{
  "postId": "a1b2c3",
  "content": "JavaScript勉強中!",
  "author": {
    "name": "Shake",
    "isVerified": true
  },
  "tags": ["programming", "js"],
  "likes": 120
}
```

- **投稿全体** `{ }`: 1つのデータとしてまとまっています。
- **投稿者** `author`: その中にもう一段「名前」や「認証済みか」という情報のまとまりが入っています。
- **マトリョーシカ構造**: オブジェクトを重ねることで、関連する情報を整理して持ち運ぶことができます。

5-2. プロパティの省略記法 (Shorthand)

変数名とプロパティ名が同じ場合、`:` 以降を省略できます。

```
const name = "Shake";
const role = "Admin";

// 昔
// const user = { name: name, role: role };

// 今 (スッキリ!)
const user = { name, role };
```

6. ハンズオン：ロジックの組み合わせ

これまでの知識を組み合わせてみましょう。
「生徒リストから、合格者だけの名前リストを作る」処理を書いてみましょう。

```
const students = [
  { name: "Aさん", score: 85 },
  { name: "Bさん", score: 40 },
  { name: "Cさん", score: 90 },
];

// 1. filterで合格者(60点以上)だけに絞る
// 2. mapで「名前だけの配列」に変換する
const passedNames = ________________________;
console.log(passedNames); // ["Aさん", "Cさん"]
```

答え（例）

```
const students = [
  { name: "Aさん", score: 85 },
  { name: "Bさん", score: 40 },
  { name: "Cさん", score: 90 },
];

// 1. filterで合格者(60点以上)だけに絞る
// 2. mapで「名前だけの配列」に変換する
const passedNames = students.filter((s) => s.score >= 60).map((s) => s.name);
console.log(passedNames); // ["Aさん", "Cさん"]
```

Section 02 まとめ

このセクションで、**「動的なWebアプリ」**を作るための道具が完全に揃いました。

1. **Logic**: 三項演算子とショートサーキットで、表示を自在に操る。
2. **Modern**: アロー関数とモダン演算子(`?.` `??`)で、安全かつ簡潔に書く。
3. **Data**: 配列メソッド(`map`, `filter`)とオブジェクト操作(分割代入, `spread`)で、データを華麗に扱う。

次回は、いよいよこれらを使ってコンポーネントを作る準備、**JavaScriptの総仕上げ**に入ります。